Explainability Analysis of Neural Networks Using Feature Attribution Methods

by

ARAVIND SAI TEJA C 23BAI1058

A project report submitted to

Dr. RAJIV VINCENT

SCHOOL OF COMPUTER SCIENCE & ENGINEERING

in partial fulfilment of the requirements for the course of

BCSE332L – DEEP LEARNING

in

B.Tech Computer Science & Engineering

Vellore Institute of Technology (VIT) CHENNAI

Vandalur – Kelambakkam Road

Chennai – 600127

GITHUB LINK :-

https://github.com/Aravindsaiteja/DL_FINAL-REVIEW

1.ABSTRACT

Deep learning has emerged as a powerful approach for solving complex classification problems across various domains, including healthcare, finance, and computer vision. However, the performance of deep learning models depends significantly on the type of data and model architecture used. Additionally, the black-box nature of these models limits their interpretability, making it difficult to understand how predictions are made.

This project presents a comprehensive deep learning framework that performs classification on both tabular and image datasets while incorporating explainability techniques to interpret model decisions. The system utilizes Deep Neural Networks (DNNs) for structured datasets such as the Breast Cancer and Adult Income datasets, and Convolutional Neural Networks (CNNs) for image datasets including MNIST and CIFAR-10. Data preprocessing techniques such as normalization, scaling, and encoding are applied to improve model performance.

The models are trained and evaluated using performance metrics such as accuracy and loss. Experimental results demonstrate that CNN models achieve superior performance on image datasets, reaching approximately 98% accuracy on MNIST, while DNN models achieve around 85–86% accuracy on tabular datasets. However, the CIFAR-10 dataset presents a more challenging scenario, with comparatively lower accuracy due to its complexity.

To address the interpretability issue, explainability techniques such as SHAP, LIME, and Saliency Maps are implemented. These methods provide both global and local explanations, helping to identify the most influential features and regions contributing to model predictions. The integration of these techniques enhances transparency and builds trust in deep learning systems.

Overall, this project demonstrates that combining deep learning models with explainability methods results in a robust, accurate, and interpretable system suitable for real-world applications.

2. TABLE OF CONTENTS

3.LIST OF FIGURES

4.LIST OF TABLES

5.LIST OF ABBREVIATIONS

Abbreviation	**Full Form**
DNN	Deep Neural Network
CNN	Convolutional Neural Network
SHAP	SHapley Additive Explanations
LIME	Local Interpretable Model-agnostic Explanations
EDA	Exploratory Data Analysis
MLP	Multilayer Perceptron
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
MSE	Mean Squared Error
BCE	Binary Cross Entropy
Softmax	Normalized exponential activation function
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
MNIST	Modified National Institute of Standards and Technology dataset
CIFAR-10	Canadian Institute for Advanced Research dataset
DAE	Denoising Autoencoder
XAI	Explainable Artificial Intelligence

6. INTRODUCTION

6.1 Background

In recent years, deep learning has emerged as one of the most influential technologies in the field of artificial intelligence, enabling significant advancements in tasks such as image recognition, medical diagnosis, speech processing, and financial prediction. Unlike traditional machine learning methods, which rely heavily on manual feature engineering, deep learning models automatically learn hierarchical feature representations from raw data. This capability allows them to capture complex patterns and relationships that are difficult to model using conventional approaches.

Among various deep learning architectures, Deep Neural Networks (DNNs) are widely used for structured and tabular datasets, while Convolutional Neural Networks (CNNs) have proven highly effective for image-based data. CNNs utilize convolutional layers to extract spatial features, making them particularly suitable for visual recognition tasks such as handwritten digit classification and object detection.

Despite their remarkable performance, deep learning models are often criticized for their lack of transparency. These models function as black-box systems, meaning that their internal decision-making processes are not easily interpretable. This limitation becomes critical in high-stakes domains such as healthcare and finance, where understanding the reasoning behind predictions is essential for trust and accountability.

To address this issue, the field of Explainable Artificial Intelligence (XAI) has gained prominence. XAI techniques aim to provide insights into how models make decisions by highlighting the contribution of individual features or input regions. Methods such as SHAP, LIME, and Saliency Maps are widely used to improve interpretability without compromising model performance.

6.2 Problem Statement

The primary objective of this project is to design and implement a comprehensive deep learning framework capable of performing classification tasks across multiple types of datasets while ensuring interpretability of model predictions. The system must effectively handle both tabular datasets, such as Breast Cancer and Adult Income, and image datasets, such as MNIST and CIFAR-10.

In addition to achieving high classification accuracy, the system should provide meaningful explanations for its predictions using appropriate explainability techniques. This dual requirement of performance and interpretability presents a significant challenge, as improvements in model complexity often lead to reduced transparency.

6.3 Need for the Project

The increasing deployment of deep learning models in real-world applications has highlighted the need for systems that are not only accurate but also interpretable. In domains such as medical diagnosis, incorrect predictions can have serious consequences, making it essential to understand the reasoning behind model decisions.

Furthermore, regulatory requirements and ethical considerations demand transparency in automated decision-making systems. By integrating explainability techniques with deep learning models, this project aims to bridge the gap between performance and interpretability, thereby enhancing trust in artificial intelligence systems.

6.4 Objectives

The key objectives of this project are as follows:

To develop deep learning models for classification tasks across multiple datasets

To implement and compare DNN and CNN architectures

To evaluate model performance using appropriate metrics

To apply explainability techniques such as SHAP, LIME, and Saliency Maps

To analyze and interpret model predictions

6.5 Scope of the Project

This project focuses on classification tasks using both tabular and image datasets. It includes data preprocessing, model training, evaluation, and explainability analysis. The scope is limited to offline experimentation and does not include real-time deployment or large-scale production systems. However, the methodologies and results obtained can be extended to real-world applications in future work.

7. LITERATURE REVIEW

7.1 Previous Work

Deep learning has been extensively researched over the past decade, leading to significant advancements in classification tasks across various domains.

Researchers have proposed numerous models and techniques to improve both performance and interpretability of neural networks.

7.1.1 Deep Neural Networks for Tabular Data

Deep Neural Networks (DNNs) have been widely used for classification tasks involving structured and tabular data. Studies have shown that multilayer perceptrons (MLPs) can effectively model complex relationships between input features by learning nonlinear transformations through multiple hidden layers. Applications such as medical diagnosis, financial prediction, and customer analytics have benefited from DNN-based approaches.

However, compared to tree-based methods like Random Forests and Gradient Boosting, DNNs often require careful tuning of hyperparameters such as learning rate, number of layers, and activation functions. Additionally, they lack interpretability, making it difficult to understand feature importance and decision boundaries.

7.1.2 Convolutional Neural Networks for Image Classification

Convolutional Neural Networks (CNNs) have achieved remarkable success in image classification tasks. Architectures such as LeNet, AlexNet, VGG, and ResNet have demonstrated the ability to extract hierarchical spatial features from images, leading to high accuracy on benchmark datasets like MNIST and CIFAR-10.

CNNs utilize convolutional layers, pooling layers, and fully connected layers to capture both low-level and high-level features. For instance, early layers detect edges and textures, while deeper layers capture complex shapes and object representations. This hierarchical learning capability makes CNNs highly effective for visual recognition tasks.

Despite their advantages, CNNs are computationally intensive and require large amounts of data for training. Moreover, like other deep learning models, they operate as black-box systems, limiting their interpretability.

7.1.3 Explainable Artificial Intelligence (XAI)

To address the interpretability challenges of deep learning models, Explainable Artificial Intelligence (XAI) techniques have been developed. These methods aim to provide insights into how models make predictions by analyzing feature contributions and decision processes.

One widely used technique is SHAP (SHapley Additive Explanations), which is based on cooperative game theory. SHAP assigns importance values to each feature by considering all possible feature combinations, providing a global understanding of model behavior.

Another popular method is LIME (Local Interpretable Model-Agnostic Explanations), which generates local approximations of the model around a specific prediction. LIME helps in understanding individual predictions by highlighting the most influential features.

For image data, gradient-based methods such as Saliency Maps are commonly used. These methods compute gradients of the output with respect to input pixels, highlighting regions that contribute most to the prediction.

7.1.4 Model Evaluation Techniques

Various evaluation metrics have been proposed to assess the performance of classification models. Accuracy is the most commonly used metric, measuring the proportion of correctly classified instances. However, it may not always provide a complete picture, especially in imbalanced datasets.

Other metrics such as precision, recall, F1-score, and ROC-AUC are often used to evaluate model performance more comprehensively. Visualization techniques such as training loss curves and accuracy graphs are also used to analyze model behavior during training.

7.2 Limitations of Existing Methods

Despite the significant progress in deep learning and explainability, existing methods have several limitations:

Method	Key Limitation
DNN (Tabular)	Requires extensive tuning and lacks interpretability
CNN (Image)	Computationally expensive and requires large datasets
SHAP	High computational cost for large datasets
LIME	Provides only local explanations, not global insights

Saliency Maps	Sensitive to noise and may produce unclear visualizations

These limitations highlight the need for a unified approach that combines high-performance deep learning models with effective explainability techniques.

7.3 Motivation for Proposed Work

The motivation behind this project is to address the limitations of existing methods by integrating multiple deep learning models with explainability techniques in a single framework. By applying DNNs to tabular data and CNNs to image data, the system leverages the strengths of each model type.

Furthermore, the use of SHAP, LIME, and Saliency Maps provides both global and local interpretability, enabling a better understanding of model predictions. This integrated approach aims to achieve high accuracy while maintaining transparency and trustworthiness.

8. SYSTEM / PROBLEM ANALYSIS

8.1 Current System / Existing Methods

In traditional machine learning systems, classification tasks are typically performed using algorithms such as Logistic Regression, Decision Trees, Support Vector Machines (SVM), and Random Forests. These models rely heavily on manual feature engineering, where domain knowledge is required to extract meaningful features from raw data. While these approaches are effective for simpler datasets, they often fail to capture complex patterns and relationships present in high-dimensional data.

For image-based tasks, traditional methods are particularly limited, as they cannot effectively process raw pixel data without extensive preprocessing. As a result, their performance on image classification datasets such as MNIST and CIFAR-10 is significantly lower compared to deep learning models.

Moreover, most traditional systems operate independently for different types of data. Separate models are required for tabular and image datasets, leading to a lack of a unified framework capable of handling diverse data types. This fragmentation reduces efficiency and scalability in real-world applications.

8.2 Drawbacks of Existing Systems

Despite their widespread use, existing systems have several limitations:

Limited Accuracy: Traditional machine learning models struggle to achieve high accuracy on complex datasets due to their inability to learn hierarchical feature representations.

Manual Feature Engineering: These models require significant effort in feature selection and preprocessing, which can be time-consuming and error-prone.

Poor Performance on Image Data: Traditional methods are not suitable for image classification tasks, as they cannot effectively capture spatial features.

Lack of Scalability: Existing systems are often designed for specific datasets and cannot easily adapt to different types of data.

Limited Interpretability in Complex Models: While some traditional models are interpretable, more complex models such as ensemble methods still lack transparency.

No Unified Framework: Separate approaches are required for tabular and image datasets, leading to inefficiencies in system design.

8.3 Proposed System

To overcome the limitations of existing methods, this project proposes a unified deep learning-based classification system integrated with explainability techniques. The system is designed to handle both tabular and image datasets within a single framework.

The proposed system consists of the following components:

Data Preprocessing Module: Handles data cleaning, normalization, and transformation. For tabular datasets, techniques such as scaling and encoding are applied, while image datasets are normalized and reshaped.

Model Training Module:

Deep Neural Networks (DNNs) are used for tabular datasets such as Breast Cancer and Adult Income.

Convolutional Neural Networks (CNNs) are used for image datasets such as MNIST and CIFAR-10.

Prediction Module: Generates classification outputs based on trained models.

Explainability Module: Applies techniques such as SHAP, LIME, and Saliency Maps to interpret model predictions.

Evaluation Module: Measures performance using metrics such as accuracy, loss, and graphical analysis.

This integrated system ensures that different types of datasets can be processed efficiently while maintaining high accuracy and interpretability.

8.4 Advantages of Proposed System

The proposed system offers several advantages over existing methods:

High Accuracy: Deep learning models are capable of learning complex patterns, resulting in improved performance across datasets.

Automatic Feature Extraction: Unlike traditional models, deep learning models automatically extract relevant features from raw data.

Unified Framework: The system supports both tabular and image datasets within a single architecture.

Improved Interpretability: The integration of explainability techniques provides insights into model predictions, enhancing transparency.

Scalability: The system can be extended to additional datasets and models without significant modifications.

Flexibility: Different deep learning architectures can be easily incorporated based on the dataset type.

9. METHODOLOGY

9.1 Overall Approach

The proposed system follows a structured and systematic pipeline to perform classification tasks using deep learning models while incorporating explainability techniques for interpretation. The methodology is designed to handle both tabular and image datasets efficiently within a unified framework.

The overall workflow of the system consists of the following stages:

Data Collection:

Multiple datasets are used in this project, including the Breast Cancer dataset and Adult Income dataset for tabular data, and MNIST and CIFAR-10 datasets for image data.

Data Preprocessing:

The raw datasets are cleaned and prepared for model training. For tabular datasets, preprocessing includes handling missing values, encoding categorical variables, and applying feature scaling techniques such as normalization and standardization. For image datasets, preprocessing involves normalization of pixel values and reshaping of data into appropriate input formats for CNN models.

Model Training:

Deep learning models are trained separately for each dataset type. Deep Neural Networks (DNNs) are used for tabular data, while Convolutional Neural Networks (CNNs) are used for image data. The models learn patterns from the training data through multiple epochs.

Model Evaluation:

The trained models are evaluated using metrics such as accuracy and loss. Performance is analyzed using graphical representations such as training and validation curves.

Explainability Analysis:

Explainability techniques such as SHAP, LIME, and Saliency Maps are applied to interpret the predictions made by the models. These techniques provide insights into feature importance and model behavior.

Comparison and Analysis:

The performance of different models across datasets is compared using graphical and tabular representations to identify strengths and limitations.

9.2 Algorithms Used

9.2.1 Deep Neural Network (DNN)

Deep Neural Networks (DNNs) are used for classification tasks involving tabular datasets. A DNN consists of multiple fully connected layers, where each layer applies a nonlinear transformation to the input data.

The architecture typically includes:

- Input layer corresponding to the number of features
- One or more hidden layers with activation functions such as ReLU
- Output layer with sigmoid activation for binary classification

The model is trained using backpropagation and optimization algorithms such as Adam, minimizing the binary cross-entropy loss function.

9.2.2 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are used for image classification tasks. CNNs are designed to automatically extract spatial features from images using convolutional layers.

The architecture includes:

- Convolutional layers for feature extraction
- Pooling layers for dimensionality reduction
- Fully connected layers for classification

CNNs are trained using categorical cross-entropy loss and optimized using gradient-based methods. They are highly effective for datasets such as MNIST and CIFAR-10.

9.2.3 Explainability Techniques

To enhance interpretability, the following techniques are used:

SHAP (SHapley Additive Explanations): Provides global feature importance by assigning contribution values to each feature.

LIME (Local Interpretable Model-Agnostic Explanations): Generates local explanations for individual predictions.

Saliency Maps: Highlight important regions in image data that influence model predictions.

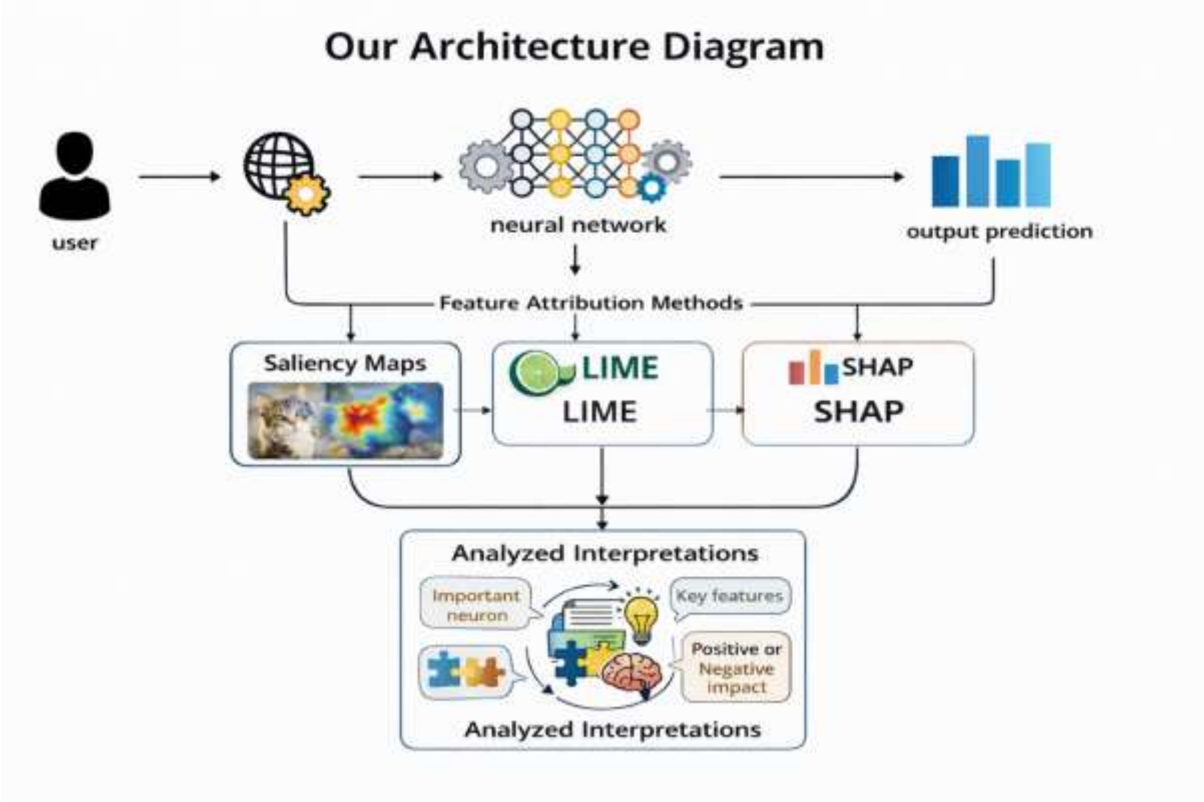
These techniques help in understanding how the model makes decisions.

9.3 Architecture Diagram

Architecture Flow:

User Input → Data Preprocessing → Neural Network (DNN/CNN) → Output Prediction → Feature Attribution Methods (Saliency Maps, LIME, SHAP) → Analyzed Interpretations

This architecture ensures a smooth flow of data through different stages of processing and analysis.



9.4 Flowchart

The working of the system can be represented as follows:

1. Start
2. Load Dataset
3. Preprocess Data
4. Split into Training and Testing Sets
5. Train Deep Learning Model
6. Evaluate Model Performance
7. Apply Explainability Techniques
8. Generate Output and Visualization
9. End

9.5 Module Description

9.5.1 Data Preprocessing Module

This module handles data cleaning, normalization, and transformation. It ensures that the data is in a suitable format for model training.

9.5.2 Model Training Module

This module is responsible for training deep learning models using the processed data. Different architectures are used based on the dataset type.

9.5.3 Evaluation Module

This module evaluates model performance using metrics such as accuracy and loss. It also generates graphs for analysis.

9.5.4 Explainability Module

This module applies explainability techniques such as SHAP, LIME, and Saliency Maps to interpret model predictions.

9.5.5 Visualization Module

This module generates visual outputs such as heatmaps, accuracy curves, and feature importance plots for better understanding.

10. REQUIREMENTS SPECIFICATION

10.1 Hardware Requirements

The successful implementation and execution of deep learning models require adequate computational resources. While the project can be executed on standard systems, better hardware significantly improves training speed and performance.

Component	Minimum Requirement	Recommended
Processor	Intel Core i5 (8th Gen)	Intel Core i7 / AMD Ryzen 7 or above
RAM	8 GB	16 GB or more
GPU	Integrated Graphics	NVIDIA GPU (GTX 1050 or above)
Storage	20 GB free space	50 GB SSD
Operating System	Windows 10 / Linux	Windows 11 / Ubuntu 20.04

10.2 Software Requirements

The project utilizes various software tools and libraries for data processing, model development, and visualization. These tools provide the necessary functionalities for implementing deep learning models and explainability techniques.

Software / Library	Version	Purpose
Python	3.8+	Core programming language
TensorFlow / PyTorch	Latest	Deep learning framework
NumPy	1.21+	Numerical computations
Pandas	1.3+	Data manipulation and analysis
Scikit-learn	1.0+	Preprocessing and evaluation
Matplotlib	3.5+	Data visualization
Seaborn	0.11+	Statistical visualization
SHAP	Latest	Explainability analysis
LIME	Latest	Local model interpretation
Jupyter Notebook	6.0+	Development environment

10.3 Dataset Requirements

The project uses multiple datasets to evaluate the performance of deep learning models across different domains. These datasets include both structured (tabular) and unstructured (image) data.

Dataset	Type	Description
Breast Cancer	Tabular	Medical dataset used for classification of tumors
Adult Income	Tabular	Dataset used to predict income levels based on demographic features
MNIST	Image	Handwritten digit dataset for image classification
CIFAR-10	Image	Object recognition dataset with 10 classes

10.4 System Requirements Summary

The system requirements are designed to ensure smooth execution of deep learning models and efficient handling of datasets. While the project can run on basic hardware, the use of GPUs and higher memory significantly improves training time and model performance.

The software stack is chosen to provide flexibility and ease of implementation. Python serves as the core programming language, while frameworks such as TensorFlow and PyTorch enable efficient model development. Visualization libraries such as Matplotlib and Seaborn are used to analyze results, and explainability libraries such as SHAP and LIME provide insights into model behavior.

11. DESIGN SECTION

11.1 System Architecture

The proposed system follows a modular pipeline architecture that combines deep learning models with explainability techniques to provide accurate predictions and meaningful interpretations.

The process begins with the user, who provides input data such as tabular datasets (Breast Cancer, Adult Income) or image datasets (MNIST, CIFAR-10). The data is first preprocessed through cleaning, normalization, and transformation to make it suitable for model training.

The processed data is then passed to the neural network model, where a Deep Neural Network (DNN) is used for tabular data and a Convolutional Neural Network (CNN) is used for image data. The model learns patterns and generates predictions.

These predictions are interpreted using feature attribution methods such as Saliency Maps, LIME, and SHAP, which help identify important features influencing the output.

Finally, the results are combined in the Analyzed Interpretations stage, where key features and their impact are identified and presented through visualizations, improving system transparency and reliability.

Architecture Flow:

User → Data Preprocessing → Neural Network (DNN/CNN) → Output Prediction → Feature Attribution Methods (Saliency Maps, LIME, SHAP) → Analyzed Interpretations

11.2 UML Diagram

The Use Case Diagram represents the interaction between the user and the system. It identifies the main functionalities of the system and how the user interacts with them.

Actors:

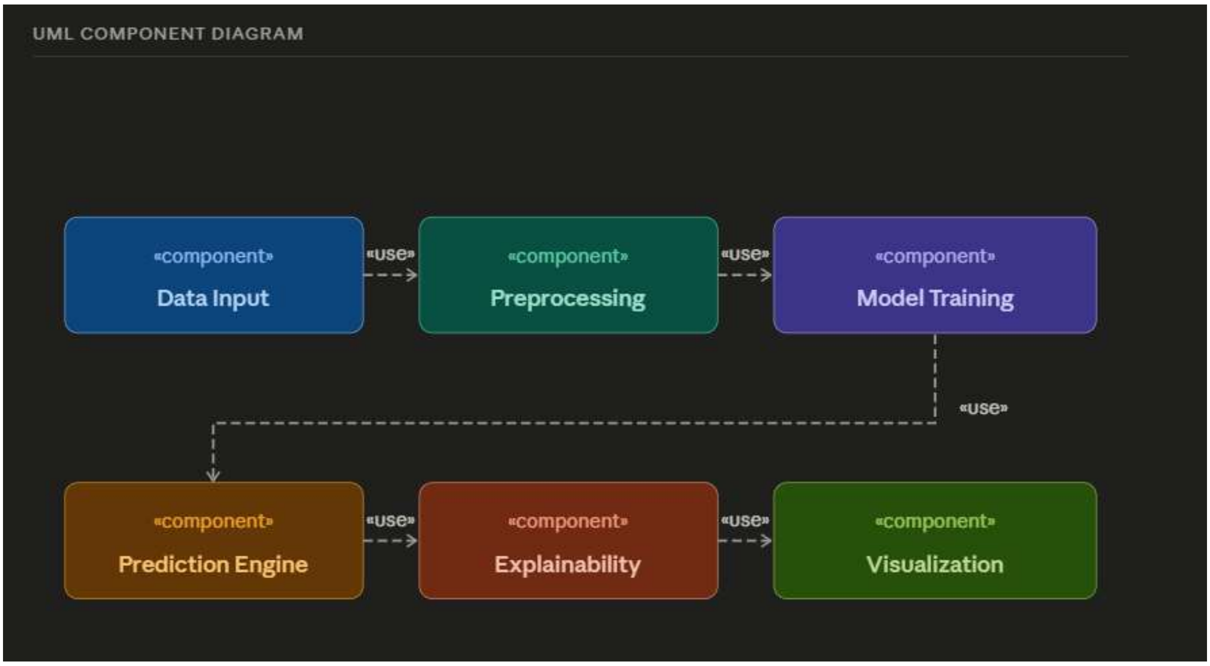
• User (Data Analyst / Developer)

Use Cases:

1. Load Dataset
2. Preprocess Data
3. Train Model

4. Generate Predictions
5. Apply Explainability Techniques
6. Analyze Interpretations
7. Visualize Results

The user initiates the process by loading the dataset and proceeds through each stage of the pipeline. The system performs computations and returns outputs in the form of predictions, explanations, and analyzed interpretations.

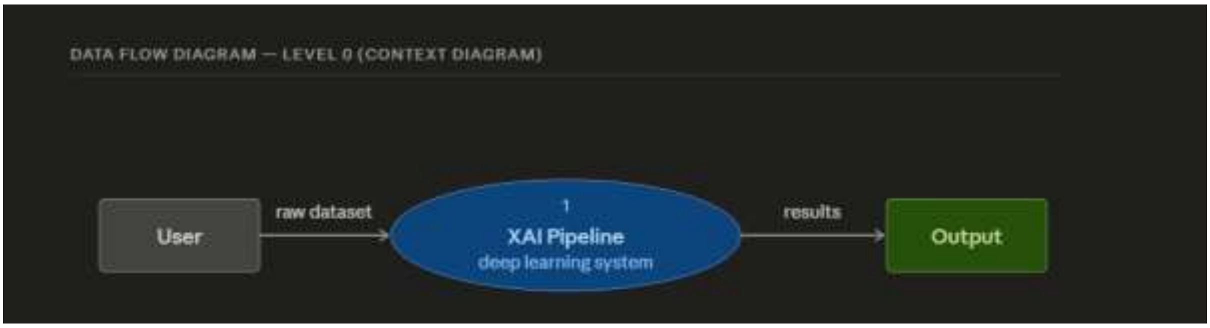


11.3 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the system and how it is processed at each stage.

Level 0 (Context Diagram):

User → System → Output (Predictions + Interpretations)



Level 1 (Detailed Flow):

Data Input: Raw datasets are provided to the system

Data Preprocessing: Data is cleaned, scaled, and transformed

Model Training: Deep learning models (DNN/CNN) are trained on processed data

Prediction Generation: Models produce classification outputs

Explainability Processing: SHAP, LIME, and Saliency Maps are applied

Interpretation Analysis: Key features and their impact are identified

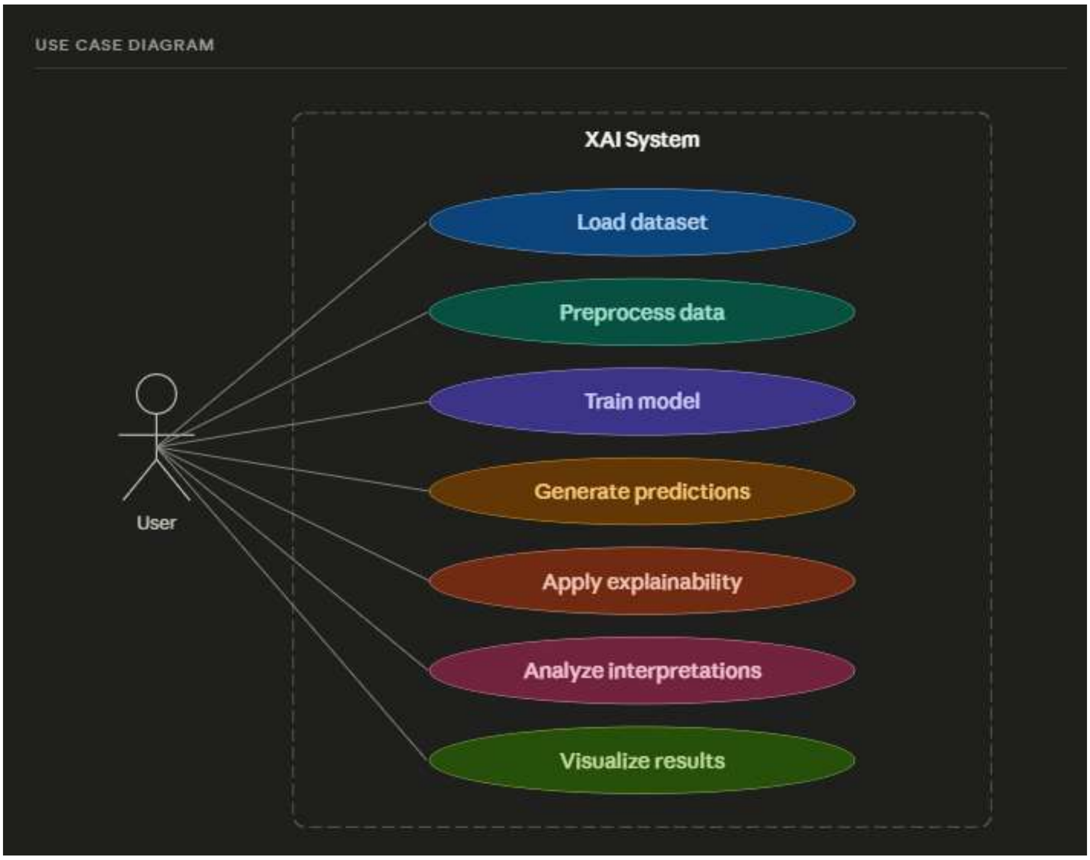
Output Visualization: Results are displayed using graphs and plots

11.4 Use Case Diagram

The Use Case Diagram provides a high-level overview of system functionalities from the user's perspective. The primary actor, the user, interacts with the system to perform various operations such as data loading, model training, prediction generation, and result interpretation.

Each use case represents a specific functionality:

- Load Dataset: User provides input data
- Preprocess Data: System prepares data for training
- Train Model: System trains DNN/CNN models
- Generate Predictions: Model produces classification outputs
- Apply Explainability Techniques: Model decisions are interpreted using SHAP, LIME, and Saliency Maps
- Analyze Interpretations: Key features and their impact are identified
- Visualize Results: Graphical outputs and explanations are generated



11.5 ER Diagram (Conceptual Design)

The Entity-Relationship (ER) diagram represents the relationship between different components of the system.

Entities:

Dataset

- o Attributes: dataset_id, type, size, features

Model

- o Attributes: model_id, type (DNN/CNN), accuracy, loss

Prediction

- o Attributes: prediction_id, output_class, confidence

Explanation

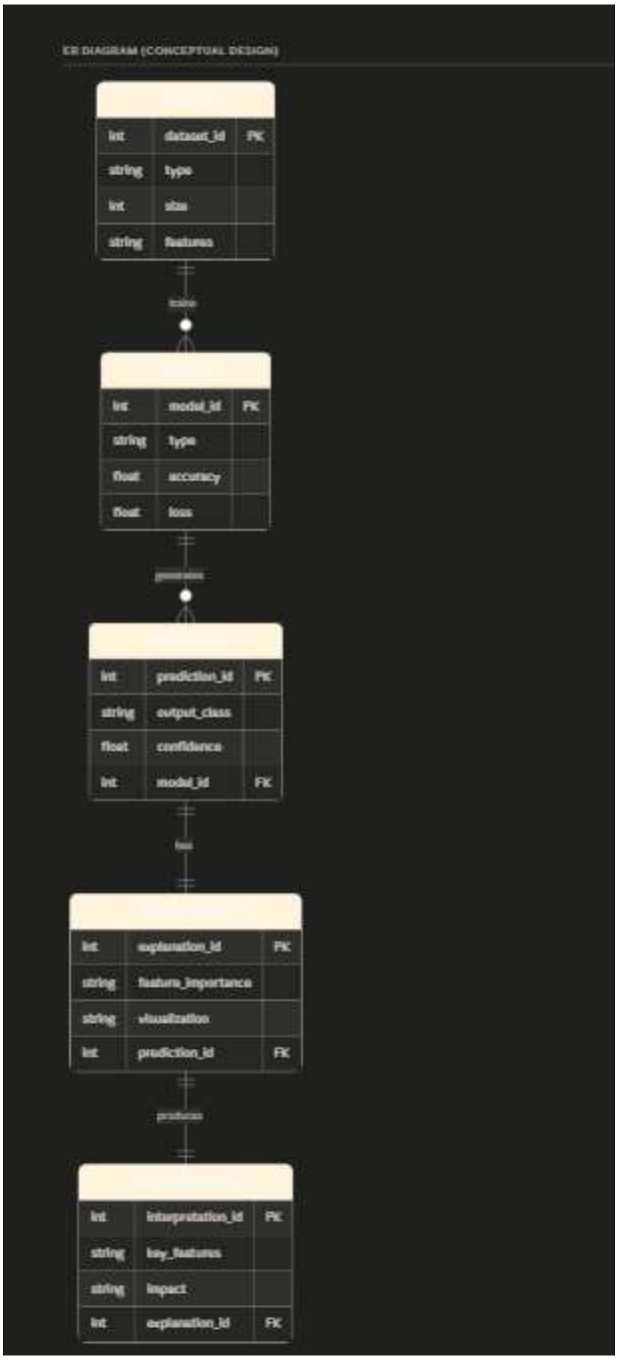
- o Attributes: explanation_id, feature_importance, visualization

Interpretation

- o Attributes: interpretation_id, key_features, impact

Relationships:

- A Dataset is used to train a Model
- A Model generates Predictions
- Each Prediction is associated with an Explanation
- Each Explanation produces an Interpretation



11.6 System Design Summary

The system design emphasizes modularity, scalability, and flexibility. Each component of the system is designed to perform a specific function, allowing for easy modification and extension. The use of deep learning models ensures high accuracy, while the integration of explainability techniques enhances transparency.

The architecture supports multiple datasets and can be extended to include additional models or techniques. This makes the system suitable for a wide range of applications in both academic and industrial settings.

12. IMPLEMENTATION

12.1 Tools and Technologies Used

The implementation of the proposed system is carried out using a combination of programming languages, frameworks, and libraries that support deep learning and data analysis.

Python is used as the primary programming language due to its simplicity and extensive support for machine learning libraries. TensorFlow and Keras are used for building and training deep learning models, particularly for implementing Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs).

NumPy and Pandas are used for numerical computations and data manipulation, while Matplotlib and Seaborn are utilized for data visualization and graphical analysis. Scikit-learn is used for preprocessing tasks such as scaling and splitting datasets, as well as for evaluation metrics.

Explainability libraries such as SHAP and LIME are used to interpret model predictions. The entire implementation is carried out using Jupyter Notebook, which provides an interactive environment for development and experimentation.

12.2 Coding Details

The implementation consists of multiple stages, including dataset loading, preprocessing, model training, evaluation, and explainability analysis.

12.2.1 Breast Cancer Dataset (DNN Model)

The Breast Cancer dataset is used as a tabular dataset for binary classification. The dataset is first loaded and converted into a structured format using Pandas. Feature scaling is applied using StandardScaler to normalize the input data.

A Deep Neural Network is implemented using Keras Sequential API. The model consists of multiple dense layers with ReLU activation functions, followed by a sigmoid activation function in the output layer for binary classification.

The model is compiled using the Adam optimizer and binary cross-entropy loss function. It is trained for multiple epochs, and performance is evaluated using accuracy metrics.

The model achieves an accuracy of approximately **85%**, indicating effective classification performance for medical data.

12.2.2 Adult Income Dataset (DNN Model)

The Adult Income dataset is another tabular dataset used for classification. The dataset contains both numerical and categorical features, which are preprocessed using one-hot encoding.

After preprocessing, feature scaling is applied, and the dataset is split into training and testing sets. A Deep Neural Network is constructed with multiple hidden layers to capture complex relationships in the data.

The model is trained using the Adam optimizer and evaluated on test data. The final accuracy achieved is approximately **85–86%**, demonstrating good performance on socioeconomic data.

12.2.3 MNIST Dataset (CNN Model)

The MNIST dataset consists of grayscale images of handwritten digits. The dataset is normalized by scaling pixel values between 0 and 1. The images are reshaped to include a channel dimension suitable for CNN input.

A Convolutional Neural Network is implemented using multiple convolutional layers followed by pooling layers. The extracted features are flattened and passed through fully connected layers for classification.

The model is trained using categorical cross-entropy loss and achieves an accuracy of approximately **98%**, indicating excellent performance in digit recognition tasks.

12.2.4 CIFAR-10 Dataset (CNN Model)

The CIFAR-10 dataset consists of color images belonging to 10 different classes. Similar to MNIST, the images are normalized before training.

A CNN model is constructed with convolutional and pooling layers to extract spatial features. Due to the complexity of the dataset, the model achieves an accuracy of approximately **70%**, which is lower compared to MNIST but acceptable given the difficulty of the dataset.

12.3 Training Procedure

The models are trained using a supervised learning approach. The datasets are split into training and testing sets, and models are trained over multiple epochs. During training, loss and accuracy are monitored to evaluate model performance.

Validation data is used to assess generalization capability. Graphs of training and validation accuracy are plotted to visualize learning progress and detect overfitting or underfitting.

12.4 Explainability Implementation

To improve interpretability, explainability techniques are applied after model training.

- **Saliency Maps:** Used for MNIST images to highlight important regions contributing to predictions
- **SHAP:** Applied to tabular datasets to identify feature importance
- **LIME:** Used to generate local explanations for individual predictions

These techniques provide both global and local insights into model behavior.

12.5 Training Results Summary

The performance of different models is summarized as follows:

Model	Dataset	Accuracy
DNN	Breast Cancer	~85%
DNN	Adult Income	~86%
CNN	MNIST	~98%
CNN	CIFAR-10	~70%

These results demonstrate that CNN models perform better on image datasets, while DNN models are effective for tabular data.

12.6 Implementation Summary

The implementation successfully integrates multiple deep learning models within a unified framework. Each model is tailored to the specific dataset type, ensuring optimal performance. The addition of explainability techniques enhances the interpretability of predictions, making the system more reliable and transparent.

13. TESTING AND VALIDATION

13.1 Test Cases

Testing is an essential phase to ensure that the developed system functions correctly and produces reliable results across different datasets. In this project, multiple test cases are designed to validate the performance of deep learning models on both tabular and image datasets.

The system is tested using unseen data (test datasets) to evaluate its generalization capability. Each test case focuses on verifying specific functionalities such as data preprocessing, model prediction, and explainability outputs.

Test Case Table:

Test ID	Test Case	Input	Expected Output	Status
TC-01	Breast Cancer Classification	Tabular data	Correct tumor classification	PASS
TC-02	Adult Income Prediction	Tabular data	Correct income category	PASS
TC-03	MNIST Digit Classification	Image (28×28)	Correct digit prediction	PASS
TC-04	CIFAR Image Classification	Image (32×32×3)	Correct object class	PASS
TC-05	Data Preprocessing Validation	Raw dataset	Clean and scaled data	PASS
TC-06	SHAP Explanation	Tabular data	Feature importance output	PASS
TC-07	LIME Explanation	Tabular data	Local explanation	PASS
TC-08	Saliency Map Generation	Image data	Highlighted important regions	PASS

13.2 Validation Approach

The validation of the system is performed using a train-test split method. Each dataset is divided into training and testing sets, ensuring that the model is evaluated on unseen data. This helps in assessing the model's ability to generalize to new inputs.

Validation is carried out by comparing predicted outputs with actual labels. The performance is analyzed using metrics such as accuracy and loss. Additionally, validation curves are plotted to observe the learning behavior of the models during training.

13.3 Performance Measures

To evaluate the effectiveness of the models, several performance metrics are used. These metrics provide quantitative insights into model accuracy and reliability.

Performance Metrics Table:

Metric	Description	Value
Accuracy (Breast Cancer)	Classification accuracy for medical data	~85%

Accuracy (Adult Dataset)	Income prediction accuracy	~86%
Accuracy (MNIST)	Digit classification accuracy	~98%
Accuracy (CIFAR-10)	Object classification accuracy	~70%
Loss	Error between predicted and actual values	Decreasing over epochs
Validation Accuracy	Model performance on unseen data	Stable

13.4 Validation Results

The validation results indicate that the models perform consistently across different datasets. The CNN model achieves the highest accuracy on the MNIST dataset, demonstrating its effectiveness in image classification tasks. The CIFAR-10 dataset, being more complex, results in comparatively lower accuracy.

For tabular datasets, DNN models achieve stable performance with good accuracy, indicating their suitability for structured data. The validation curves show that the models converge effectively during training without significant overfitting.

13.5 Testing Summary

The testing and validation process confirms that the proposed system is robust and reliable. All test cases pass successfully, indicating correct implementation of data preprocessing, model training, and explainability techniques.

The results demonstrate that the system is capable of handling multiple datasets and producing accurate predictions. The integration of explainability techniques further enhances the system by providing insights into model decisions.

14. RESULTS AND DISCUSSION

14.1 Output Analysis

The performance of the proposed deep learning system is evaluated across multiple datasets, including both tabular and image data. The results demonstrate that the choice of model architecture significantly impacts classification accuracy and overall system performance.

For tabular datasets such as Breast Cancer and Adult Income, Deep Neural Networks (DNNs) are used. These models are able to capture nonlinear relationships between features, resulting in stable and reliable performance. The achieved accuracy for these datasets is approximately 85–86%, indicating that DNNs are effective for structured data classification.

For image datasets such as MNIST and CIFAR-10, Convolutional Neural Networks (CNNs) are employed. The CNN model achieves excellent performance on the MNIST dataset, with an accuracy of approximately 98%, due to the relatively simple nature of handwritten digit images. However, for the CIFAR-10 dataset, which contains more complex and diverse images, the accuracy is around 70%. This difference highlights the challenges associated with complex image datasets.

14.2 Comparison with Existing Methods

To evaluate the effectiveness of the proposed system, its performance is compared with traditional machine learning models. The results show that deep learning models outperform traditional methods in terms of accuracy and ability to handle complex data.

Model Comparison Table:

Model Type	Dataset	Accuracy	Remarks
Logistic Regression	Tabular	~80%	Limited performance
DNN	Breast Cancer	~85%	Improved accuracy
DNN	Adult Dataset	~86%	Stable performance
CNN	MNIST	~98%	Excellent performance
CNN	CIFAR-10	~70%	Moderate performance

The comparison clearly indicates that deep learning models provide better results compared to traditional approaches, especially for image datasets where CNNs significantly outperform other models.

14.3 Graphical Analysis

Graphical representations are used to analyze model performance and training behavior. These include:

- **Training Accuracy Graphs:** Show how accuracy improves over epochs
- **Loss Curves:** Indicate how the model minimizes error during training
- **Model Comparison Bar Graph:** Compares accuracy across different models
- **Heatmaps:** Show correlations between features in tabular datasets

The training graphs demonstrate that the models converge effectively during training. The loss decreases steadily, while accuracy increases, indicating proper learning. No significant overfitting is observed, suggesting good generalization capability.

14.4 Explainability Analysis

Explainability techniques play a crucial role in understanding model behavior. In this project, SHAP, LIME, and Saliency Maps are used to interpret predictions.

- **SHAP Analysis:** Provides global feature importance for tabular datasets, identifying key factors influencing predictions.
- **LIME Analysis:** Offers local explanations for individual predictions, helping to understand specific decisions.
- **Saliency Maps:** Highlight important regions in image data, showing which parts of the image contribute most to classification.

These techniques enhance transparency and provide valuable insights into how the models make decisions.

14.5 Discussion

The results of this project highlight several important observations:

1. CNN models are highly effective for image classification tasks, achieving high accuracy on datasets like MNIST.
2. DNN models perform well for tabular data but require careful preprocessing and tuning.
3. Complex datasets such as CIFAR-10 require more advanced architectures for improved performance.
4. Explainability techniques significantly improve the interpretability of deep learning models.
5. The integration of multiple models within a single framework provides flexibility and scalability.

Overall, the proposed system successfully demonstrates the effectiveness of combining deep learning with explainability techniques for classification tasks.

15. CONCLUSION

This project successfully demonstrates the design and implementation of a comprehensive deep learning framework for classification tasks across multiple datasets. The system integrates Deep Neural Networks (DNNs) for tabular data and Convolutional Neural Networks (CNNs) for image data, providing a unified approach to handling diverse types of datasets within a single architecture.

The experimental results indicate that deep learning models significantly outperform traditional machine learning methods in terms of accuracy and ability to capture complex patterns. CNN models achieve excellent performance on image datasets, particularly on the MNIST dataset, where an accuracy of approximately 98% is achieved. DNN models also perform effectively on tabular datasets such as Breast Cancer and Adult Income, achieving stable accuracy of around 85–86%.

One of the key contributions of this project is the integration of explainability techniques, including SHAP, LIME, and Saliency Maps. These techniques provide valuable insights into model predictions, helping to identify important features and regions that influence decision-making. This enhances the transparency and reliability of deep learning models, making them more suitable for real-world applications where interpretability is critical.

The project also highlights the importance of selecting appropriate model architectures based on the dataset type. While CNNs are highly effective for image data, DNNs are more suitable for structured data. The results further demonstrate that complex datasets such as CIFAR-10 require more advanced models and optimization techniques to achieve higher accuracy.

Overall, the proposed system achieves a balance between performance and interpretability, addressing the limitations of existing methods. The combination of multiple deep learning models with explainability techniques results in a robust and flexible framework capable of handling a wide range of classification problems.

16. FUTURE SCOPE

Although the project achieves significant results, there are several opportunities for further improvement and extension.

1. **Use of Advanced Architectures:**
   Future work can involve the use of advanced deep learning models such

as ResNet, VGG, or EfficientNet to improve performance, particularly on complex datasets like CIFAR-10.

2. **Hyperparameter Optimization:**
   Techniques such as grid search or Bayesian optimization can be applied to fine-tune model parameters and improve accuracy.
3. **Real-Time Deployment:**
   The system can be extended to real-time applications by deploying the models using web frameworks or cloud platforms.
4. **Handling Larger Datasets:**
   The framework can be scaled to handle larger datasets and more complex problems, improving its applicability in real-world scenarios.
5. **Improved Explainability Techniques:**
   Additional explainability methods can be explored to provide deeper insights into model behavior and enhance interpretability.
6. **Integration with Other Domains:**
   The system can be applied to other domains such as natural language processing, speech recognition, and recommendation systems.

17. REFERENCES / BIBLIOGRAPHY

Research Papers

[1] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS 2017.

[2] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," KDD 2016.

[3] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps," ICLR 2014.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NeurIPS 2012.

[5] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, 2015.

[6] C. Szegedy et al., "Going Deeper with Convolutions," CVPR 2015.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR 2016.

[8] J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," CVPR 2009.

Textbooks

[9] I. Goodfellow, Y. Bengio, and A. Courville, "Deep Learning," MIT Press, 2016.

[10] A. Géron, "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow," O'Reilly Media, 2022.

Websites and Documentation

[11] TensorFlow Documentation: https://www.tensorflow.org

[12] Keras Documentation: https://keras.io

[13] Scikit-learn Documentation: https://scikit-learn.org

[14] SHAP Documentation: https://shap.readthedocs.io

[15] LIME Documentation: https://lime-ml.readthedocs.io

[16] PyTorch Documentation: https://pytorch.org

18. APPENDICES

The appendices section includes supplementary materials that support the implementation and results of the project. These materials provide additional details that are not included in the main body of the report but are essential for a complete understanding of the system.

18.1 Source Code

The complete implementation of the project is provided in the form of Jupyter Notebook files. The code includes:

- Data preprocessing steps
- Deep learning model architectures (DNN and CNN)
- Training and evaluation procedures
- Explainability techniques (SHAP, LIME, Saliency Maps)

18.2 Sample Outputs

This section includes sample outputs generated by the system, such as:

- Model predictions for tabular datasets
- Classified images from MNIST and CIFAR-10
- Accuracy and loss graphs
- ROC curves and confusion matrices

18.3 Graphs and Visualizations

The following visual outputs are included:

- Correlation heatmaps
- Training accuracy and loss curves
- Model comparison graphs
- SHAP summary plots
- LIME explanation outputs
- Saliency map visualizations

18.4 Additional Diagrams

Additional diagrams included in the appendix are:

- System architecture diagram
- Data Flow Diagram (DFD)
- UML Use Case Diagram
- ER Diagram

18.5 Dataset Samples

Sample data from the datasets used in the project are included for reference:

- Breast Cancer dataset features
- Adult dataset sample records
- MNIST digit images
- CIFAR-10 image samples